MIMIC-IV QA Benchmark: Implementation

1 Phase 1: Corpus Construction

1a. Condition selection. We restrict the corpus to conditions likely to produce multi-cluster candidate pools. ICD-10 codes are grouped at the 3-character prefix level (e.g. `I50` = heart failure, `E11` = type 2 diabetes) rather than used at full granularity. Full ICD-10 codes are highly specific (e.g. `I50.1` vs. `I50.20`) and fragment admissions into hundreds of tiny buckets; grouping at the prefix level gives clinically coherent disease categories with enough admissions to compute comorbidity statistics. Conditions are then ranked by $N_{\text{admissions}} \times c^2$, where c is the mean number of active Charlson comorbidity categories per admission (out of 17 high-level disease groups: myocardial infarction, congestive heart failure, diabetes, renal disease, cancer, etc.). For the experiment, only the top 8 diseases from the output are used.
Output: `conditions_stats.parquet` (`icd10_3char`, `condition_name`, `n_admissions`, `mean_comorbidity_count`, `score`).

1b. Note chunking. We parse the discharge notes and keep only high-value sections (Brief Hospital Course, History of present illnesses, Pertinent Results, Discharge Diagnosis). The Brief Hospital Course gets the most attention: complex admissions structure it as a `\#`-delimited problem list, so we split on those markers to produce one chunk per active clinical issue. Simple admissions with no `\#` markers produce a single BHC chunk. Each chunk has a minimum of 15 tokens and a maximum of 512 tokens (if it's bigger than that, it gets chunked with a stride of 128 tokens).
Output: `chunks.parquet` (`chunk_id`, `note_id`, `hadm_id`, `subject_id`, `section_name`, `text`, `chief_complaint`, `approx_tokens`).

1c. Admission metadata. We build a per-admission table of demographics, top-5 ICD diagnoses, and Charlson comorbidity flags. Used when the metadata per-admission is needed.
Output: `admissions_metadata.parquet` (`hadm_id`, `subject_id`, `age`, `gender`, `race`, `admissio n_type`, `primary_icd_code`, `primary_icd_description`, `top_icd_descriptions`, one binary column per Charlson category).

1d. Deduplication. Sections like Past Medical History and Allergies are often copied verbatim across a patient's repeated admissions. Deduplicate and keep one copy per (patient, section, content).
Output: `chunks.parquet` updated in place.

2 Phase 2: Embeddings

Raw chunks are too patient-specific to match condition-level queries. We prepend a structured prefix to each chunk before embedding: age group, sex, race, primary diagnosis, chief complaint, and active comorbidities names.
Output: LanceDB vector table (all chunk columns + `vector`, a float32 embedding).

3 Phase 3: Query Generation and Annotation

3a. Grounded prompts. This step builds the prompts to be used in the actual queries generation in the next step. Queries must require multi-source synthesis, so e3ach query is scoped to a (condition, modifier) pair. A *modifier* is a secondary clinical axis that partitions the condition's patient population into distinguishable sub-groups; each group should form a distinct cluster in the candidate pool.
Modifiers are of two types: the top-3 co-occurring Charlson comorbidities computed dynamically

per condition and fixed demographic thresholds (age > 75, age < 40). For each (condition, modifier) pair, real chunk excerpts from patients at the condition+modifier intersection are injected into the prompt, grounding the LLM in the actual content present in the data, to generate queries that are actually answerable given the dataset. The prompt explicitly requires the question to connect ≥ 2 clinical aspects, ensuring the gold annotation will produce a multi-facet structure.
Output: `queries_prompts.parquet` (`icd10_3char`, `condition_name`, `modifier_text`, `modifier_type`, `n_intersection_admissions`, `grounding_hadm_ids`, `full_prompt`).

3b. Query generation. Each prompt is sent to a local LLM to produce the question text. Earlier runs used simpler queries and produced no measurable signal, so the queries that are produced in this run are complicated and span different aspects (all medical though)
Output: `queries.parquet` (same as above minus `full_prompt`, plus `query_text`).

3c. Divergence pre-filter. We keep only queries where coverage diverges: for each query we run both methods on a cosine top-N pool from the full corpus (N = 3000 in the experiment) and compute Jaccard divergence and Facility-Location score. We keep only queries for which the jaccard similarity is under a ceretain thershold.
Output: `divergence_stats.parquet` (queries columns + `jaccard_div`, `fac_gap`, `fac_topk`, `fac_fl`, `pool_size`, `passes_filter`).

3d. Gold annotation (map-reduce LLM). For each filtered query, a local LLM (gemma4:31b) reads the condition-filtered candidate pool in batches and produces (`fact`, `facet_label`, `chunk_ids`) triples. Facet labels are LLM-generated. A reduce step merges batch results into a final (facet $\rightarrow$ chunk IDs) mapping.
Note: This candidate pool over which the gold annotation runs is filtered exactly by condition+modifier metadata, it's not a top-N cosine similarity result. This is intentional, since only these filtered chunks will contain gold answers. The evaluation step won't run on the filtered chunks, but will run on a restricted candidate pool (3000 chunks in this run) retrieved by cosine similarity over the whole corpus.
Output: `gold_annotations.parquet` (`query_id`, `icd10_3char`, `condition_name`, `modifier_text`, `query_text`, `facets_json`, `n_facets`, `n_gold_chunks`).

4 Phase 4: Evaluation

Candidate pool. For each query the pool is the top-N (N is prefilter-n: 3000 in the yaml configs) chunks from the full corpus by cosine similarity, unrestricted by condition. This is deliberate: the pool must contain the redundant dominant cluster (many similar notes on the primary condition) and the complementary clusters (notes from patients with different comorbidities). Also, running the facility-location on the whole corpus of chunks is computationally heavy, so this retrieval is actually a re-ranking step after retrieving the top-N (with N large enough) by ordinary cosine similarity.

Strategies. Top-k (baseline), MMR (dispersion), and facility-location (coverage) are compared at each (k, λ) with $k \in \{5, 10, 20, 30\}$ and $\lambda \in \{0.3, 0.5, 0.7, 0.8\}$.

Metrics.

- **Aspect Recall (AR)**: for each gold facet, asks: did we retrieve at least one chunk that covers it? AR is the fraction of facets for which the answer is yes. A score of 1.0 means every distinct clinical aspect has at least one representative in the retrieved set.
- **Weighted Aspect Recall (WAR)**: same as AR but instead of a binary hit/miss per facet, it measures how completely each facet is covered (retrieved gold chunks / total gold chunks for that facet), then averages across facets.

- **Gold Precision (GP)**: of the k retrieved chunks, what fraction are gold?
- **Gold Recall (GR)**: of the gold chunks that are actually present in the candidate pool, what fraction did we retrieve?
- **FL coverage score (fac)**: the facility-location objective evaluated on the retrieved set.
- **Jaccard vs. top-k (jac)**e gain comes from marginal additions.

Outputs:

- `evaluation_results.parquet` (one row per query × strategy × k ×λ): `query_id`, `strategy`, `k`, `lam`, `aspect_recall`, `weighted_aspect_recall`, `gold_precision`, `gold_recall`, `fac_cov_score`, `avg_cos`, `jaccard_vs_topk`, `n_unique_hadms`)
- `evaluation_stats.parquet` (means of all metrics aggregated by strategy/k/λ).

---

5 Experiment Results (n = 13 queries)

Setup. 8 conditions (ICD-10 3-char prefixes) × up to 5 modifiers each = 40 generated queries; 13 passed the divergence filter (Jaccard threshold = 0.8 at $k = 10$, $\lambda = 0.35$). Gold annotation used a condition-filtered top-900 cosine pool (Phase 3d); evaluation used the full-corpus top-3000 cosine pool. Embedding model: `multi-qa-mpnet-base-cos-v1`.

Example query. Condition: *Type 2 diabetes mellitus* (`E11`), modifier type: comorbidity, modifier: *chronic kidney disease* (Charlson category `renal_disease`).
Generated question: "*In patients with Type 2 diabetes and advanced kidney disease, how do clinicians manage the transition between pharmacological management of glycemic control and the initiation of renal replacement therapy, such as dialysis, and what specific lab values or symptoms typically trigger these changes?*" Gold facets: `glycemic_management_transition` and `rrt_initiation_triggers` ($n_{\text{gold}} = 133$ chunks across both facets).

Table 1: Comparison of strategies across distinct k Values (given k, only the top performing λ value is reported for MMR and FL)

k	Strategy	AR	WAR	GP	GR	jac	fac
5	top-k	0.244	0.008	0.138	0.009	1.000	0.777
	FL ($\lambda = 0.5$)	0.282	0.008	0.169	0.010	0.718	0.795
	MMR ($\lambda = 0.7$)	0.301	0.010	0.154	0.010	0.320	0.772
10	top-k	0.387	0.014	0.138	0.019	1.000	0.795
	FL ($\lambda = 0.7$)	0.372	0.013	0.131	0.018	0.904	0.802
	MMR ($\lambda = 0.7$)	0.462	0.013	0.115	0.013	0.260	0.791
20	top-k	0.576	0.024	0.142	0.029	1.000	0.815
	FL ($\lambda = 0.3$)	0.621	0.026	0.150	0.031	0.796	0.827
	MMR ($\lambda = 0.8$)	0.669	0.023	0.135	0.028	0.535	0.815
30	top-k	0.751	0.035	0.149	0.040	1.000	0.826
	FL ($\lambda = 0.5$)	0.751	0.034	0.144	0.039	0.971	0.829
	MMR ($\lambda = 0.8$)	0.800	0.032	0.136	0.037	0.538	0.827

Why failure? FL beats top-k at $k = 5$ and $k = 20$, but falls below top-k at $k = 10$ (0.372 vs. 0.387) and ties at $k = 30$. The failure at $k = 10$: with a pool of 3000 and only 10 selections, FL's coverage objective allocates picks to represent the full pool geometry, but maybe the pool contains many near-condition non-gold chunks. At $k = 20$ the budget is large enough for FL to

cover the high-similarity region and still reach complementary facets. At $k = 30$, top-k finds most facets by sheer volume, and aggressive coverage ($\lambda = 0.3$) actively hurts (AR= 0.736 < top-k = 0.751) by forcing picks into the pool periphery.

Annotation vs. evaluation mismatch. Gold Precision and Recall are very low (GP$\approx$ 0.10–0.17 and GR$\leq$ 0.04), why? The annotation pool was the condition-filtered top-900 cosine pool; the evaluation pool is the full-corpus top-3000. Condition-specific gold chunks may rank below 3000 when retrieved by the full corpus. The effect is shared by all methods. Fix: merge annotation pool and evaluation pool so no gold chunk is structurally excluded?

Large gold sets and implications for AR. Gold chunk counts range from 29 to 285 (mean $\approx$ 139) across the 13 queries, with 2–5 facets each. The exhaustive tagging design was correct in principle, but the cardinality of the gold sets reveals that each facet is "large" in embedding space, supported by many chunks across a lot of admissions. The coverage advantage of FL is expected to be more pronounced in the full-scale run where facets may be tighter and query–pool structure clearer, and where the embedding model can be biomedical (e.g. `BioBERT` or `MedCPT`) to sharpen clinical semantic separation.

Issues with AR. FL at its best λ has high Jaccard overlap with top-k (jac= 0.90 at $k = 10$; 0.80 at $k = 20$): FL selects mostly the same chunks as top-k plus a few from the nearest uncovered pool region, and the AR gain comes from those marginal additions. MMR at high λ diverges substantially more (jac$\approx$ 0.26–0.54) and achieves higher AR, meaning the chunks that most improve facet coverage happen to be distant ones that MMR reaches maximizing dispersion. Given wide gold sets and a general-purpose embedding model, the full-corpus pool does not have the tight multi-cluster structure for coverage to win. Fix: try with a biomedical embedding model to see if embeddings distribute differently? Change candidate pools?